

Boot Image Pipeline — From Block Design to Power-On

This guide describes how the files in `release/` are produced and how the board boots on its own: how the block design becomes a bitstream, how the bootloader ELF is merged into that bitstream without re-running synthesis, how the QSPI flash is laid out, and what happens at power-on. It is the background reference for the [Standalone Boot chapter of the datasheet](#).

None of this is required to use the board. It is intended for rebuilding the boot images or understanding how the pieces fit together.

The key idea

A bitstream describes more than logic and routing: it also contains the initial contents of every block RAM on the chip. The CPU's 128 KB local memory is built from BRAM, so whatever program those initialization frames hold is present the moment the FPGA finishes configuring, before any software has run. As a result, a bootloader can be built into the hardware image, is restored at every power-on, and cannot be rendered unbootable by software. For the same reason, `updatemem` can replace a program in an already-built bitstream in seconds: only the BRAM init frames change, while logic, routing, and timing stay untouched.

Pipeline overview

```
block design
├── synthesis + implementation (Vivado, one-time)
└── ▼
    top.bit ..... logic + routing + BLANK BRAM
    │
    │ + top_wrapper.mmi (address→BRAM map, in the XSA)
    │ + bootloader ELF
    │ ▼ updatemem
    boot.bit / boot_srec.bit ..... BRAM init = bootloader
    │
    │ + application image (raw binary / SREC)
    │ ▼ write_cfgmem
    *.mcs ..... flash-shaped image
    │
    │ ▼ program_hw_cfgmem
    QSPI flash
    │
    │ ▼ power-on
    FPGA configures itself → bootloader in BRAM → app runs
```

Stage 1 — Block design → `top.bit` (Vivado, one-time)

Synthesis and implementation turn the block design into a stream of *configuration frames*: every LUT, every route, and the INIT values of every RAMB36 primitive. At this point the BRAM init values are all zeros, so `top.bit` programmed on its own leaves the CPU executing empty memory and the UART silent. `top.bit` is deliberately kept in this blank state; the bootloader is stitched in afterwards (Stage 3).

Vivado can also embed an ELF at `write_bitstream` time (ELF association). This repository does not use that flow: keeping `top.bit` generic means a bootloader change never requires re-running the implementation.

Stage 2 — `top_wrapper.mmi`: the address-to-BRAM map (Vivado, one-time)

The 128 KB local memory is not one physical block. It consists of 32 RAMB36 primitives (4 KB each) distributed across the die, each holding only a few bits of every 32-bit word. Which physical cell stores the byte at CPU address `0x1234` is known only after placement, so Vivado writes this mapping into `top_wrapper.mmi` (shipped inside `release/top_wrapper.xsa`). Any hardware rebuild changes the placement, so the MMI must always be re-extracted from the matching XSA.

Stage 3 — `updatemem`: merge an ELF into the bitstream (seconds, no re-synthesis)

```
updatemem -force -meminfo top_wrapper.mmi -data <bootloader>.elf \
-bit release/top.bit -proc top_i/microblaze_riscv_0 -out <boot>.bit
```

`updatemem` reads the ELF's loadable segments (the bootloader occupies `0x0-0x8000`), uses the MMI to scatter each byte into the correct INIT frame of the correct RAMB36, and rewrites only those frames. Logic and timing are bit-for-bit unchanged, which is why this takes ~6 seconds instead of a 20-minute implementation run. `release/boot_srec.bit` is exactly `top.bit` with the bootloader ELF merged in.

Stage 4 — `write_cfgmem`: lay out the flash image (Vivado)

The FPGA's configuration engine expects the bitstream to reside at flash offset `0x0` in a specific byte order. `write_cfgmem` produces that image, and can place additional raw data at other offsets in the same file:

```
write_cfgmem -force -format mcs -size 4 -interface SPIx1 \
-loadbit "up 0x0 boot_srec.bit" \
-loaddata "up 0x00220000 showcase.srec" \
-file official_boot.mcs
```

Flash map (4 MB QSPI)

Offset	Contents	Read by
--------	----------	---------

Offset	Contents	Read by
0x000000–0x21FFFF	Bitstream (with bootloader in BRAM init)	FPGA configuration engine, at power-on
0x220000–0x3FFFFFF	Application slot (SREC text, 1.875 MB)	srec_spi_boot loader

The slot starts immediately after the bitstream region (an uncompressed xc7a35t bitstream is at most 0x217214 bytes, so 0x220000 is a safe boundary); its offset is a bootloader configuration constant (blconfig.h).

Stage 5 — program_hw_cfgmem: write the flash over JTAG (Vivado Hardware Manager)

The FPGA provides no direct JTAG command for writing the flash. Vivado therefore first programs a temporary bridge bitstream that turns the FPGA into a JTAG ↔ SPI programmer, then erases, writes, and verifies. The important property setting is:

```
set_property PROGRAM.ADDRESS_RANGE {use_file} $cfg
```

use_file scopes the erase and program to the address ranges present in the MCS, which makes it safe to update one region without affecting the others. Afterward, boot_hw_device forces the FPGA to reconfigure from flash. This is functionally equivalent to a power cycle and allows the actual boot path to be verified without handling the board.

Stage 6 — Power-on: one flash, two readers

The same QSPI flash is read twice, by two different readers:

1. **The configuration engine.** This is dedicated silicon, present independently of the user design. With the mode pins set to Master-SPI it streams the bitstream from offset 0x0 into the chip. Once the BRAM init frames load, the bootloader is already in memory. No software is involved, so the bootloader is restored at every power-on regardless of what the previous program did.
2. **The QSPI flash controller in the design.** Once the design is running, the CPU leaves reset at address 0x0 and executes the bootloader, which uses this controller to read the application from its flash slot, copy it to SRAM, and jump to it.

The block design therefore provides the storage (BRAM) and the transfer path (flash controller); updatemem determines what the BRAM holds at power-on, and the flash layout determines what is copied to SRAM.

Which tool does what

Step	Tool	Vivado needed?
------	------	----------------

Step	Tool	Vivado needed?
Build bootloader / application ELFs	Vitis	No
<code>objcopy -O srec (app → SREC)</code>	RISC-V toolchain (ships with Vitis)	No
Stitch ELF into bitstream (<code>updatemem</code>)	CLI tool — ships with both Vitis and Vivado	No
JTAG load / debug (<code>xsdb</code> , IDE)	Vitis	No
Synthesize / implement → <code>top.bit</code> + XSA/MMI	Vivado	Yes (one-time)
Build flash image (<code>write_cfgmem</code>)	Vivado Tcl	Yes
Program flash, range-scoped (<code>program_hw_cfgmem</code> + <code>use_file</code>)	Vivado Hardware Manager	Yes

In summary, all software work requires only Vitis, and once the hardware is frozen even the ELF-into-bitstream step requires nothing from Vivado. Vivado remains necessary for the one-time hardware build and for the flash-image and programming steps.

Vitis also ships a standalone `program_flash` CLI that can write an image to a given flash offset through the FPGA — board-verified 2026-07-12 for application-slot updates, which makes a Vitis-only deployment possible. Give it the SREC file with a `.bin` extension so its bytes are written verbatim rather than parsed as an image format.

The prebuilt artifacts in `release/`

File	What it is
<code>top.bit</code>	Implementation output, blank BRAM — never boot this alone
<code>top_wrapper.xsa</code>	Hardware handoff for Vitis; contains <code>top_wrapper.mmi</code>
<code>boot_srec.bit</code>	<code>top.bit</code> + SREC bootloader (JPROGRAM = remote power-cycle)
<code>official_boot.mcs</code>	Full flash image: bitstream + bootloader + demo app SREC at <code>0x220000</code>